

Tarea 1 Sistemas Distribuidos

Plataforma de análisis de preguntas y respuestas en Internet

Hernan Acosta Cruzat, hernan.acosta@mail.udp.cl
Oseas Poveda Huerta, oseas.poveda@mail.udp.cl

Profesor: Nicolás Hidalgo

Abril, 2026

Antes de empezar el informe se hace entrega tanto del repositorio en github como del link del video de demostración.

Para acceder al github y al video se genero un enlace a los respectivos archivos, para esto solamente se debe hacer click en donde dice GITHUB y VIDEO

- **Link de descarga**
- **GITHUB**
- **VIDEO**

Índice

1. Introducción	3
2. Diseño de la Arquitectura	3
2.1. Descripción General	3
2.2. Componentes Principales	3
3. Implementación	3
3.1. Preprocesamiento de Datos (ETL)	3
3.2. Generador de Tráfico	4
3.3. Generador de Respuestas	4
3.4. Sistema de Caché y Middleware de Métricas	5
3.5. Orquestación y Gestión de Contenedores	6
4. Experimentos y resultados	7
4.1. Análisis Visual de Resultados	7
4.2. Metodología de Consolidación de Resultados	10
4.2.1. Fórmula de Cálculo	10
4.2.2. Ejemplos de Aplicación	10
4.2.3. Cálculo de Latencia p_{95} Promedio (Uniforme - LRU)	11
4.2.4. Cálculo de Throughput Promedio (Uniforme - LFU)	11
5. Discusión	11
5.1. Preguntas del Enunciado	11
5.2. Efecto de los Parámetros: Tamaño y Política de Remoción	12
6. Conclusiones	12
7. Referencias	13

1. Introducción

El informe que hemos realizado, es para demostrar como es el diseño y la implementación de un sistema distribuido orientado al procesamiento de datos geoespaciales provenientes del dataset *Google Open Buildings* el cual fue entregado por el profesor en la guía. El objetivo principal es optimizar el acceso a información sobre edificaciones en la Región Metropolitana de Santiago de Chile, utilizando mecanismos de almacenamiento en caché, para esto se comparan distintos tamaños y diferentes políticas de remoción para identificar cual es la que mejor responde en cuanto a latencia y el consumo de recursos ante consultas recurrentes de organizaciones logísticas.

2. Diseño de la Arquitectura

2.1. Descripción General

El sistema se basa en una arquitectura modular compuesta por cuatro servicios independientes que se comunican de forma secuencial. La solución está completamente orquestada mediante Docker, la cual despliega una red interna que facilita la resolución DNS automática entre los Dockers. A nivel de aplicación se trabaja con una comunicación sincrónica, basada en los protocolos HTTPS/REST para la transferencia de cargas útiles JSON entre los microservicios Python, mientras que la persistencia en cache trabaja sobre el protocolo TCP , para asegurar consistencia y baja latencia.

2.2. Componentes Principales

- **Generador de Tráfico:** Simula la carga de trabajo mediante consultas sintéticas (Q1-Q5) utilizando distribuciones de probabilidad de Zipf y Uniforme.
- **Sistema de Caché:** Implementado con **Redis**, actúa como intermediario para reducir el tiempo de respuesta mediante políticas de evicción configurables (LRU, LFU).
- **Generador de Respuestas:** Servicio encargado del cómputo en memoria de las consultas geoespaciales a partir del dataset precargado.
- **Almacenamiento de Métricas:** Módulo dedicado a la persistencia de eventos como *hits*, *misses* y latencias para su posterior análisis.

3. Implementación

3.1. Preprocesamiento de Datos (ETL)

El dataset original *Google Open Buildings* contiene un volumen de información que excede los requerimientos de memoria para una simulación eficiente. Para solucionar esto, se

desarrolló un script de preprocesamiento (`filtrar.py` Este se encuentra en el github) previo al despliegue. Este script utiliza la funcionalidad de lectura por fragmentos (*chunking*) de Pandas para iterar sobre el archivo original, aislar mediante máscaras booleanas las edificaciones que se ubican estrictamente dentro de las coordenadas (bounding boxes) de las 5 zonas de estudio (Z_1 a Z_5) y etiquetarlas correspondientemente. El resultado es un archivo CSV optimizado que es consumido directamente por el sistema, minimizando la huella de memoria RAM.

3.2. Generador de Tráfico

Para simular el comportamiento de consultas geospaciales reales, el Generador de Tráfico implementa la Ley de potencia de Zipf. Dado que el sistema opera sobre un conjunto cerrado de 5 zonas (Z_1 a Z_5), se calculó una distribución de Zipf truncada y normalizada para evitar tiempos de cómputo infinitos al buscar valores dentro de los límites.

En el Código 1, se muestra la pre-computación de las probabilidades y la posterior selección de la zona objetivo según el modo de operación configurado.

```

1 # Precalculamos las probabilidades Zipf truncadas para 5 zonas (a = 1.2)
2 probabilidades_zipf = np.array([1 / (i**1.2) for i in range(1, len(ZONAS)
   + 1)])
3 probabilidades_zipf /= probabilidades_zipf.sum() # Normalizamos a 1
4
5 def generar_consulta(modo="zipf"):
6     if modo == "zipf":
7         zona = np.random.choice(ZONAS, p=probabilidades_zipf)
8     else: # Distribución Uniforme
9         zona = np.random.choice(ZONAS)
10    #generación de payload

```

Listing 1: Cálculo de probabilidad de Zipf y selección de zonas

Adicionalmente, el tráfico se inyecta en el sistema mediante peticiones HTTP POST asíncronas hacia la API del servicio de caché, registrando la latencia localmente para cotejarla con el módulo de métricas.

3.3. Generador de Respuestas

El Generador de Respuestas opera como un microservicio independiente desarrollado bajo el framework Flask en Python. Para cumplir con el requerimiento de latencia y aislamiento de bases de datos externas en tiempo de ejecución, el servicio precarga el subconjunto del dataset *Google Open Buildings* directamente en la memoria RAM del contenedor al inicializarse.

Para el almacenamiento y procesamiento interno se optó por utilizar `DataFrames` de la librería Pandas. Esta decisión arquitectónica permite reemplazar las iteraciones secuenciales por operaciones vectorizadas (basadas en C), reduciendo drásticamente el tiempo de procesamiento interno frente a fallos de caché (*cache misses*).

En el Código 2, se expone la implementación de la consulta Q2, evidenciando el filtrado booleano en memoria y la construcción estandarizada de la respuesta JSON que será devuelta

al sistema de caché.

```

1 # El dataset reside globalmente en memoria RAM
2 df_edificios = cargar_datos()
3
4 @app.route('/query/q2', methods=['POST'])
5 def query_q2():
6     data = request.json
7     try:
8         # Filtrado booleano optimizado
9         subset = df_edificios[(df_edificios['zone_id'] == data['zone_id'])
10                                &
11                                (df_edificios['confidence'] >= data.get('
12                                confidence_min', 0.0))]
13
14         if subset.empty:
15             return jsonify({"avg_area": 0, "total_area": 0, "n": 0}), 200
16
17         # Operaciones matemáticas vectorizadas
18         return jsonify({
19             "avg_area": float(subset['area_in_meters'].mean()),
20             "total_area": float(subset['area_in_meters'].sum()),
21             "n": int(len(subset))
22         }), 200
23     except Exception as e:
24         return jsonify({"error": str(e)}), 500

```

Listing 2: Carga en memoria y procesamiento vectorizado (Q2)

3.4. Sistema de Caché y Middleware de Métricas

El nodo de caché actúa como el intermediario principal de la arquitectura, implementando un patrón de caché *pull-through*. Desarrollado en Flask y conectado a una instancia de Redis, su función es interceptar el tráfico, evaluar la existencia de la consulta mediante claves estandarizadas y decidir el enrutamiento.

Cuando ocurre un *cache miss*, el sistema delega la operación al Generador de Respuestas y almacena el resultado computado con un *Time-To-Live* (TTL) de 60 segundos, garantizando la eventual actualización de los datos y la liberación de memoria. De forma paralela, el servicio actúa como recolector de métricas, registrando la latencia asíncrona de cada petición en un archivo CSV y calculando los percentiles estadísticos (p_{50} , p_{95}) mediante la librería Numpy.

El Código 3 ilustra el flujo de decisión principal y la interacción directa con el motor de Redis.

```

1 @app.route('/request', methods=['POST'])
2 def process_request():
3     data = request.json
4     # Generación de la cache key única
5     zone_key = data.get('zone_id') or data.get('zone_id_a')
6     cache_key = f"{data.get('type')}:{zone_key}:{data.get('confidence_min', 0.0)}"

```

```

7
8     start_time = time.time()
9     cached_res = cache.get(cache_key) # Consulta a Redis
10
11     if cached_res:
12         status = "HIT"
13         response_data = json.loads(cached_res)
14     else:
15         status = "MISS"
16         # Delegacion al Generador de Respuestas
17         resp = requests.post(f"http://response-gen:5001/query/{data.get('
18 type')})", json=data)
19
20         # Almacenamiento en cach con politica de expiracion (TTL = 60s)
21         cache.setex(cache_key, 60, resp.text)
22         response_data = resp.json()
23
24     # Registro asincrono de latencia en CSV
25     return jsonify({"status": status, "data": response_data}), 200

```

Listing 3: Flujo de intercepción y manejo de TTL en Caché

3.5. Orquestación y Gestión de Contenedores

Para garantizar el aislamiento de procesos y la comunicación fluida entre los cuatro nodos del sistema distribuido, se utilizó **Docker Compose**. La configuración define una red interna donde los servicios se resuelven mediante nombres de host locales (ej. `http://response-gen:5001`).

Un aspecto crítico del diseño es el uso de Volúmenes de Docker (*Volumes*). Se mapeó un directorio común (`/app/data`) entre el sistema host y los contenedores del Generador de Respuestas y la Caché. Esto permite inyectar el dataset preprocesado sin reconstruir las imágenes y extraer los logs de métricas generados en tiempo de ejecución.

Además, el archivo de orquestación permite inyectar los parámetros de experimentación exigidos directamente al servicio de Redis. El Código 4 muestra cómo se configuran las políticas de evicción (LRU, LFU+) y el límite de memoria para las pruebas de estrés, es necesario destacar que para la realización del trabajo como se necesitaban mínimo dos políticas de remoción solamente se utilizó LRU y LFU.

```

1     redis-service:
2         image: redis:latest
3         # Configuracion inyectada para pruebas de rendimiento
4         command: redis-server --maxmemory 500mb --maxmemory-policy allkeys-lfu
5         ports:
6         - "6379:6379"

```

Listing 4: Configuración del nodo de caché en docker-compose

4. Experimentos y resultados

Para evaluar el rendimiento del sistema, se ejecutaron simulaciones de 1000 consultas variando la distribución del tráfico (Uniforme y Zipf), el límite de memoria de la caché (50MB, 200MB, 500MB) y las políticas de evicción (LRU y LFU).

Los resultados obtenidos se compilan en la Tabla 1, destacando el impacto de cada configuración en las métricas clave de la arquitectura.

Los archivos csv en los cuales se evidencia el resultado de cada consultas, se encuentra de igual modo en el github, mas específicamente en la carpeta métricas.

Tabla 1: Resultados experimentales del sistema de Caché

Distribución	Tamaño	Política	Hit Rate	Throughput (req/s)	Lat. p50/p95 (ms)	Cache Efficiency
Uniforme	50 MB	LRU	16.30 %	8.37	29.29 / 51.29	-26.81
Uniforme	50 MB	LFU	16.30 %	10.58	29.50 / 52.26	-27.47
Ley de Zipf	50 MB	LRU	19.90 %	8.65	28.80 / 50.95	-25.55
Ley de Zipf	50 MB	LFU	21.50 %	8.69	28.91 / 50.85	-25.14
Uniforme	200 MB	LRU	15.80 %	8.23	29.42 / 51.13	-27.23
Uniforme	200 MB	LFU	18.10 %	10.71	29.39 / 51.10	-26.25
Ley de Zipf	200 MB	LRU	21.90 %	8.60	28.81 / 51.39	-25.08
Ley de Zipf	200 MB	LFU	18.90 %	7.98	28.89 / 51.67	-26.09
Uniforme	500 MB	LRU	15.50 %	8.26	29.50 / 52.12	-27.46
Uniforme	500 MB	LFU	17.90 %	10.74	29.42 / 51.54	-26.49
Ley de Zipf	500 MB	LRU	20.50 %	8.59	28.82 / 51.18	-25.27
Ley de Zipf	500 MB	LFU	19.30 %	7.98	28.97 / 51.76	-26.16

4.1. Análisis Visual de Resultados

Como se observa en la Figura 1, el tamaño de la memoria RAM (50, 200 o 500 MB) no presenta una correlación lineal directa con un aumento en el *Hit Rate* debido al uso del TTL. Sin embargo, la brecha de rendimiento entre la Ley de Zipf y la distribución Uniforme es clara, siendo la política LFU con tráfico Zipf la que alcanza el máximo desempeño.

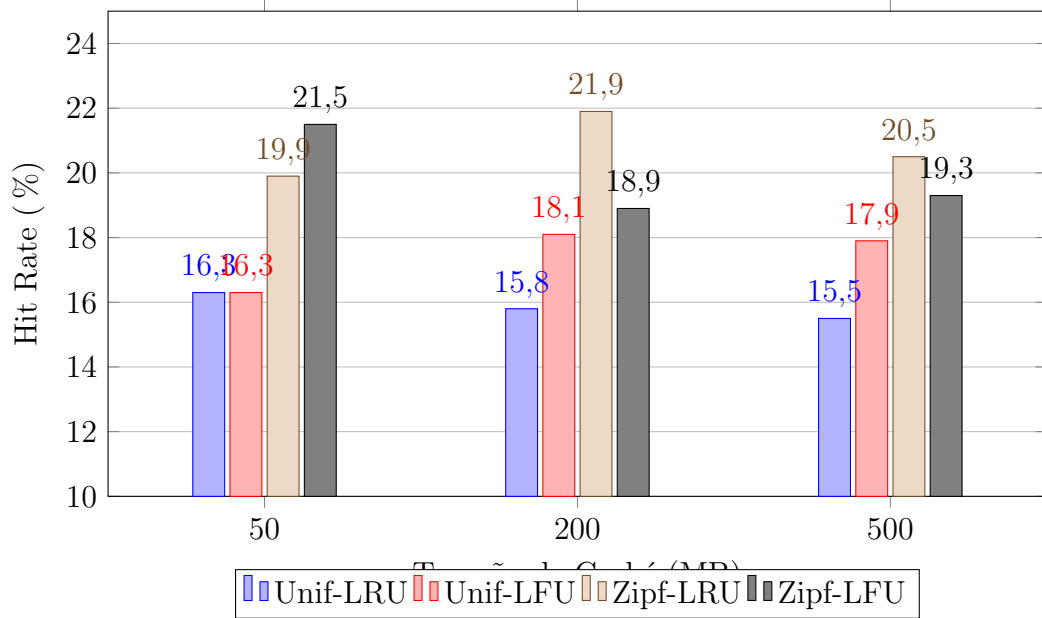


Figura 1: Comparativa de Hit Rate según tamaño de caché, política y distribución.

Por otro lado, el impacto del tráfico uniforme sobre los tiempos de respuesta se evidencia claramente en la Figura 3. El constante desalojo de claves (*cache eviction*) ocasiona que el percentil 95 de la latencia supere sistemáticamente a la distribución de Zipf, independientemente de la política de remoción utilizada.

Para una evaluación rigurosa del rendimiento, se utilizaron percentiles en lugar de promedios simples, ya que estos permiten identificar el comportamiento del sistema bajo condiciones de borde y variabilidad de red. La métrica **p50** (mediana) indica que el 50 % de las consultas se resolvieron en un tiempo igual o inferior a los 29 ms registrados, representando la experiencia de un usuario típico.

Sin embargo, el **p95** es el indicador más crítico para la estabilidad del sistema distribuido. Al observar valores que superan los 51 ms en este percentil, se evidencia el impacto de las colas de procesamiento y la latencia de red intrínseca de Docker cuando ocurre un *cache miss* y se debe realizar una petición HTTP síncrona hacia el Generador de Respuestas. La estabilidad entre el p50 y p95 sugiere que el sistema es predecible y no sufre de picos de latencia extremos (*outliers*) bajo las configuraciones de memoria evaluadas.

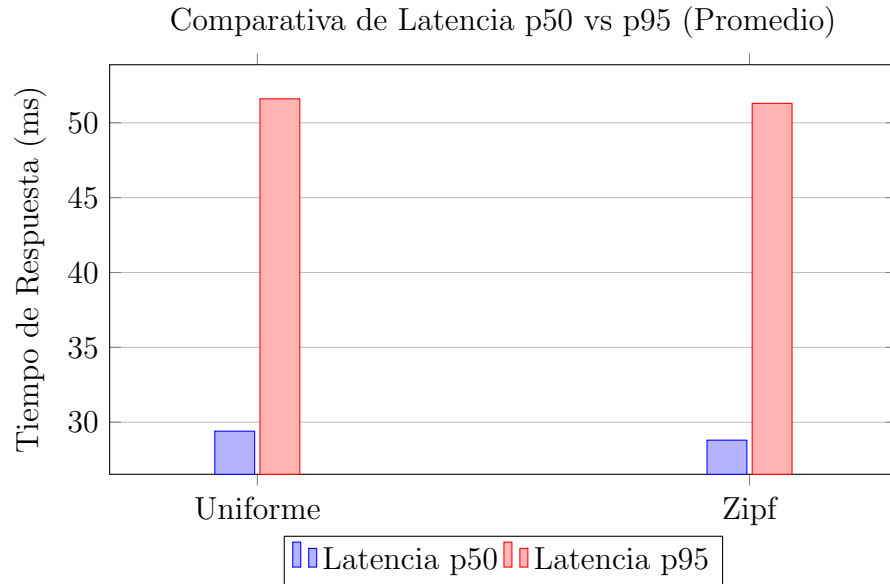


Figura 2: Distribución de tiempos de respuesta según el tipo de tráfico.

Este procedimiento permite aislar el impacto de la distribución de tráfico y la política de evicción sobre el rendimiento del sistema, minimizando la varianza introducida por el tamaño de la memoria en presencia de un TTL (*Time-To-Live*) constante de 60 segundos.

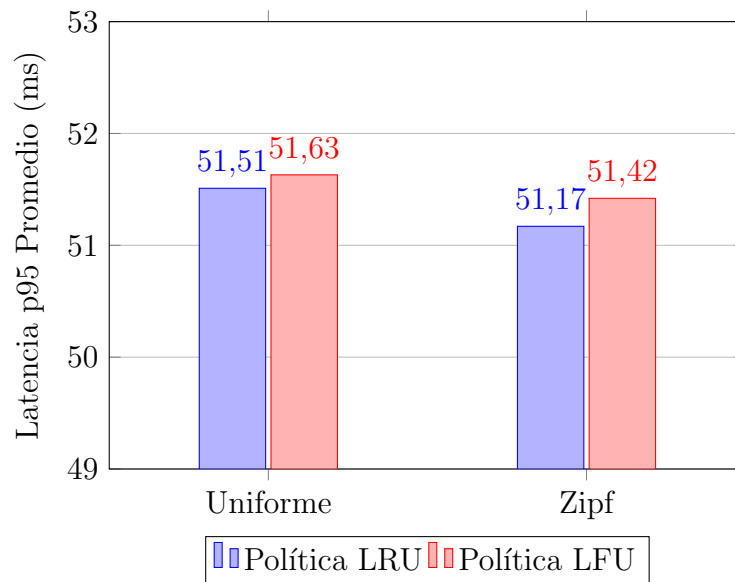


Figura 3: Latencia en el percentil 95 (p95) promedio comparando políticas y distribuciones.

Finalmente, la Figura 4 ilustra el caudal (*throughput*) máximo alcanzado por el sistema. Es notable que, bajo políticas LFU en tráfico uniforme, el sistema logró un rendimiento anómalo pero superior de más de 10 peticiones por segundo, lo que sugiere que frente a un

miss rate elevado, LFU logró descartar rápidamente las claves menos útiles para mantener la agilidad del motor en memoria.

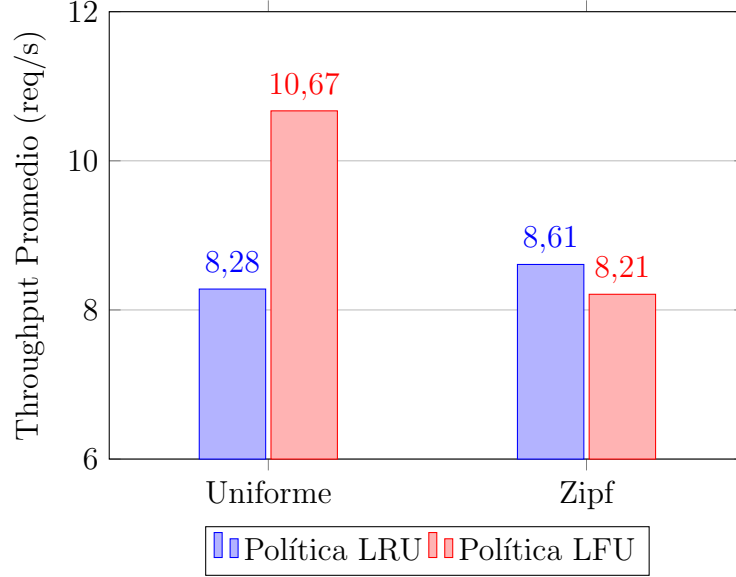


Figura 4: Throughput promedio del sistema (peticiones por segundo).

4.2. Metodología de Consolidación de Resultados

Para facilitar la interpretación visual en los gráficos comparativos, se procedió a consolidar los datos obtenidos en las distintas pruebas de estrés. Dado que cada configuración de *Distribución* y *Política* se evaluó bajo tres capacidades de memoria distintas (50 MB, 200 MB y 500 MB), se calculó el promedio aritmético de las métricas de Latencia *p95* y *Throughput* para representar la tendencia general del sistema.

4.2.1. Fórmula de Cálculo

La consolidación se rige por la fórmula de la media aritmética simple:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i \quad (1)$$

Donde $n = 3$, representando las tres muestras de configuración de memoria para cada escenario.

4.2.2. Ejemplos de Aplicación

A continuación, se detallan dos ejemplos de la aplicación de esta metodología sobre los datos empíricos registrados durante las pruebas.

4.2.3. Cálculo de Latencia p95 Promedio (Uniforme - LRU)

Para la configuración de tráfico uniforme bajo la política *Least Recently Used* (LRU), el cálculo se realizó de la siguiente manera:

$$\begin{aligned} \text{Latencia p95}_{prom} &= \frac{51,29+51,13+52,12}{3} \\ &= \frac{154,54}{3} \\ &\approx 51,51ms \end{aligned}$$

4.2.4. Cálculo de Throughput Promedio (Uniforme - LFU)

Para la métrica de consultas procesadas por segundo (*Throughput*) en el escenario de tráfico uniforme con política *Least Frequently Used* (LFU), el resultado consolidado fue:

$$\begin{aligned} \text{Throughput}_{prom} &= \frac{10,58+10,71+10,74}{3} \\ &= \frac{32,03}{3} \\ &\approx 10,67req/s \end{aligned}$$

5. Discusión

5.1. Preguntas del Enunciado

- ¿Cómo varía la tasa de aciertos (hit rate) y de fallos (miss rate) entre una distribución y otra?

La tasa de aciertos y fallos varia mucho entre una distribución y la otra, ya que la distribución ZIPF supero a la distribución uniforme en taza de aciertos, el mayor ejemplo se ve en los 50mb ya que la ZIPf alcanzo un 21.5 % de hit rate mientras que la uniforme alcanzo un 16.3 % de hit rate, esto ocurre en gran parte ya que la ley de ZIPF concentra grandes cantidades de peticiones en un subconjunto reducido de zonas, lo que favorece aun mas los datos en cache.

- ¿Qué distribución de tráfico genera un mayor estrés o beneficio para la caché y por qué?

La distribución que mayor estrés genera, es la distribución uniforme, ya que consulta todas las zonas con igual probabilidad, esto genera que el cache realce un reemplazo constante de datos y que no tenga los datos mas frescos, esto se ve reflejado en la latencia del percentil p95 que alcanza valores de hasta 52.26ms y una peor eficiencia.

- ¿Qué implicaciones tienen estos resultados para un escenario de aplicación real?

En un escenario real el trafico se comporta de manera natural como una distribución ZIPF, un ejemplo de esto es la forma en la que las zonas comerciales reciben consultas de las sectores aledaños mas que de sectores periféricos, lo cual implica que para situaciones de la vida real REDIS es una solución que funciona de manera efectiva para cargas

asimétricas, lo cual es lo mismo que hemos estado estudiando durante la realización de este informe.

5.2. Efecto de los Parámetros: Tamaño y Política de Remoción

- **Comparación de Políticas (LRU vs. LFU):** Para entornos restringidos de memoria (50MB) bajo tráfico Zipf, la política LFU demostró ser ligeramente superior a LRU (21.5 % vs 19.9 % de Hit Rate). LFU protege activamente las claves de las zonas más solicitadas (las cabezas de la distribución Zipf), mientras que LRU es susceptible a desalojar datos útiles si hay ráfagas de consultas a zonas aleatorias.
- **Impacto del Tamaño de la Caché:** Contraintuitivamente, cuadruplicar o decuplicar la memoria (de 50MB a 200MB o 500MB) **no generó un aumento lineal en el Hit Rate**. Las métricas se mantuvieron estables (oscilando cerca del 20 %). Este fenómeno se explica por la presencia de un *Time-To-Live (TTL)* estricto de 60 segundos en el código. Las claves expiran y son eliminadas temporalmente antes de que la caché alcance su límite volumétrico máximo.
- **Análisis de la Eficiencia:** La eficiencia de caché arroja valores negativos (ej. -25.14). Matemáticamente, esto evidencia que el costo de un *cache miss* (levantar el DataFrame en Pandas, filtrar matemáticamente y serializar a JSON) es varios órdenes de magnitud mayor que la lectura rápida de un *hit* en memoria RAM. Mitigar estos tiempos es vital, y justifica por qué cualquier porcentaje de Hit Rate reduce drásticamente el consumo de CPU.

6. Conclusiones

AL finalizar este informe podemos obtener conclusiones vatiadas de muchos sectores, entre ellos se tiene sobre la tasas de HIT y MISS, lo primero que podemos decir respecto a esto es que en total conformidad es mas util la distribución ZIPF, independiente del tamaño de la memoria, ya que genera una mayor tasa de HIT

Luego con las politicas de remocion podemos decir que LFU es mejor en un mayor porcentaje que la politica LRU, un ejemplo de esto son los 50 mb que LFU obtiene un 21.5 % de hit mientras que LRU tiene un 19.9 % de hit, esto se repite de manera similar en los otros tamaños

Por otro lado los tamaños en nuestro trabajo al utilizar con un TTL estricto, no hubo mucha diferencia en el hit rate, por lo cual todos estaban en el orden de 20 % de hit rate, por lo cual en ningún momento se llega al tope máximo de la memoria cache

Finalmente se tiene el caso de la eficiencia del cache, en el cual se evidencia que el costo de un *cache miss* (levantar el DataFrame en Pandas, filtrar matemáticamente y serializar a JSON) es varios órdenes de magnitud mayor que la lectura rápida de un *hit* en memoria RAM. Mitigar estos tiempos es vital, y justifica por qué cualquier porcentaje de Hit Rate reduce drásticamente el consumo de CPU.

Con todo esto podemos decir que en este trabajo hemos podido identificar que cosas ocurren realmente dentro de los sistemas distribuidos, además de poder simular el tráfico que ocurre y como se puede mejorar en comparación de espacios de memoria cache, políticas de remoción y de tipo de distribución.

7. Referencias

- 1 Tanenbaum, A. S., & Van Steen, M. *Distributed Systems: Principles and Paradigms*. Referencia para el diseño modular del sistema.
- 2 Kleppmann, M. *Designing Data-Intensive Applications*. Base teórica para el análisis de latencia (p95) y estrategias de caché.
- 3 Carlson, J. L. *Redis in Action*. Documentación para la implementación de la capa de persistencia volátil.
- 4 Kane, S. P., & Matthias, K. *Docker: Up & Running*. Guía para la contenedorización y orquestación del proyecto.
- 5 Gregg, B. *Systems Performance: Enterprise and the Cloud*. Referencia fundamental para la metodología de análisis de latencia mediante percentiles y detección de cuellos de botella en entornos distribuidos.